CYBR 437 Secure Coding

Fall 2026

Lab - 4

Release date: October 15, 2026

Due date: October 21, 2026, at 11:59 pm

Introduction

This lab will be a primer on the use of regular expressions (regex). Part 1 presents you with a number of common problems that might be solved using regular expressions in order to practice both the syntax and pattern format. Use Regex101 to test your patterns against a set of valid and invalid inputs. Once complete, add your final solution to the included `cybr437regex.py` file and add a set of 5 valid (passing) matches and 5 invalid (failing) matches to the test file `cybr437regextesting.py`. Run the test to confirm that your pattern matches each test case appropriately – include the output in your submission.

Part 2 of the lab is a practical application of your newfound understanding with a focus on vulnerability assessment using open-source static-analysis tools. Use the following tools, discussed in lecture, to analyze the provided source code: `cppcheck`, `flawfinder`, `splint`, `clang static analyzer`. Once you have identified possible vulnerabilities to target, you will write your own mock-assessment tool using RegEx to identify vulnerable/deprecated functions in the provided C source files.

Note: Make sure you document any cases you could not get to work, or any shortcomings for a given problem in part 1.

Part 1 (60 points)

Problem 1 (20 points):

Social Security Number (SSN) (can be with dashes, whitespaces, or no spaces at all)

NOTE: dashes and whitespace will only occur between places 3 and 4 and/or between places 5 and 6.

The rules are:

a) No valid SSN has all zeros in any of the 3 segments
b) No valid SSN has 9 identical digits or has the 9 digits running consecutively from 1-9
c) SSA will issue SSNs with the digits “1”, “8”, and “9” only in position 1
d) SSA will not issue SSNs beginning with the digit “9” unless the position 4 digit is a 9
e) SSA will not issue SSNs beginning with the digits “666” in positions 1-3
f) The SSN must be exactly 9 digits

Valid Examples	Invalid Examples
132456789	123-45-6789
132-45-6789	12-345-6789
143 45 6789	123--45-6789
132-45 6789	123.45.6789
135 45-6789	111-11-1111
13345-6789	000-26-6781

Problem 2 (20 points):

E-mail address – must contain a local-part and domain, separated by the @ symbol. The local-part of the e-mail address must have at least 1 character, and a minimum of 4 characters must be in the domain. While RFC 5322 allows for special characters, we will follow the generally accepted convention, limiting allowed special characters to hyphens (-), underscores (_), and the dot (.). A dot may be used in the local-part of the e-mail address, before the @ symbol, but it cannot be the first character in the string or immediately precede the @ symbol. A dot cannot be used more than once consecutively in the e-mail string. These restrictions do not apply to the hyphen or underscore.

Valid Examples	Invalid Examples
j0hn.smith@ewu.edu	john.smith.@ewu.edu
John-Smith@ewu.edu	.john..smith@ewu.edu
-john_smith-@ewu.edu	j.smith@e.u

Problem 3 (20 points):

A password that contains at least 10 characters and includes at least one uppercase character, one lowercase character, one digit, one punctuation mark, and does not have more than 3 consecutive lowercase characters or two digits that repeat. The password must start with a lowercase character or a punctuation mark.

Valid Examples	Invalid Examples
!3tsSeeIf&	paassWordd
@on3Two\"hr0	p@sSW()rds
&&&&&&&&&sS3	N0LOWERC4S3!
t3sTingM33!	only9ch@R
th!$H4$&()	Pa$sword1212

Part 2 (40 points)

Task 1 (15 points):

Analyze the source file `task1.c` with the 4 tools mentioned above and explain the outputs obtained from each of them. Also, provide a comparative analysis of the outputs. Include relevant screenshots of outputs in your submission.

Task 2 (25 points):

Write a small Python program that uses regex to detect the use of vulnerable/deprecated functions in C source files. The program should output a message for each occurrence of such a function, the line number where it occurs, and the line of code for the user. Use your analyzer to find the vulnerable functions in the source file `task2.c`.

Submission Instructions

1. Your version of the two Python files for Part 1.
2. Your code for Task 2 in Part 2.
3. Your write-up submission, which should include:
    - The regular expression pattern solution for each problem
        - Including a detailed explanation of how each part of your pattern exactly handles the requirements of the problem.
        - The 5 passing tests and 5 failing tests you used for each task and why (what they accomplished), along with screenshots of the test and output.
    - Detailed explanation of the analysis output for Task 1 in Part 2 with relevant screenshots.
    - Detailed explanation of your Python code and output screenshots for Task 2 in Part 2.